



CI1056 – ALGORITMOS E ESTRUTURA DE DADOS II

Profa. Rachel Reis
rachel@inf.ufpr.br



AULA DE HOJE

Construção de heaps

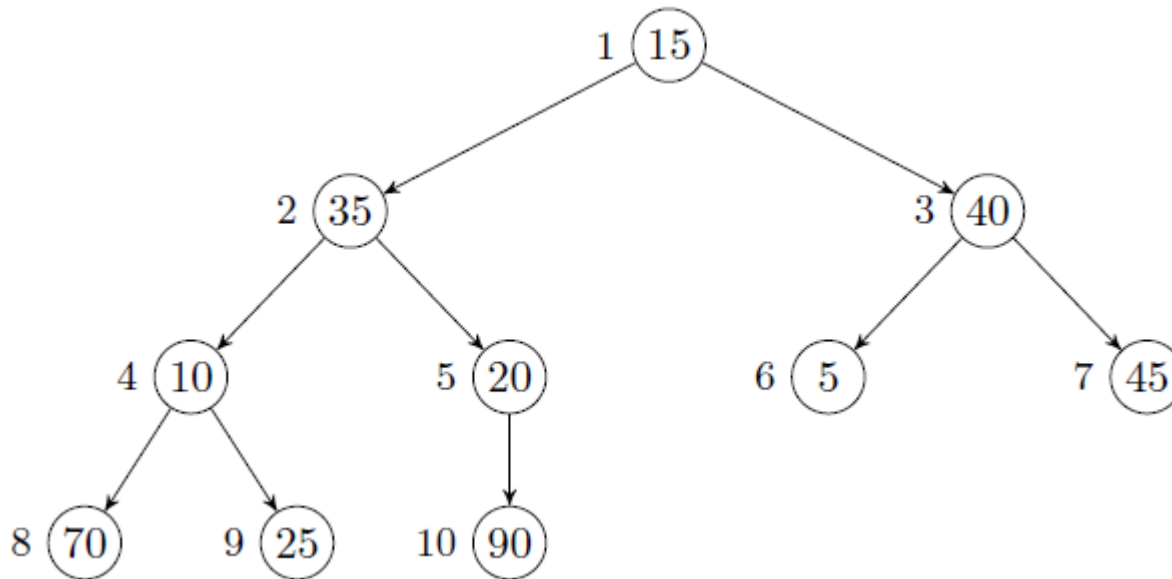
Adaptação do material gentilmente
disponibilizado pelos Profs. André Vignatti e
Marcos Castilho

MOTIVAÇÃO

- Um ***heap*** binário pode ser representado por um vetor.
- Nessa aula, vamos aprender como transformar um vetor qualquer em um *heap* de máximo.
- Isso será usado no algoritmo de ordenação Heapsort.

- Dado o vetor abaixo, como ele pode ser visto na forma de uma árvore binária (quase) completa?

1	2	3	4	5	6	7	8	9	10
15	35	40	10	20	5	45	70	25	90



É um heap?

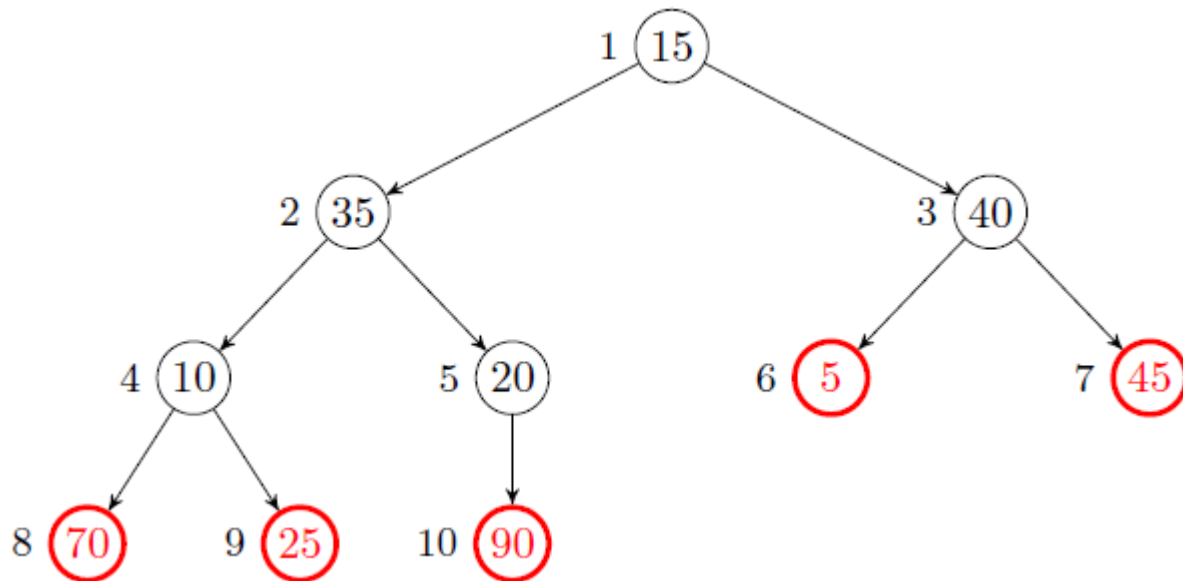
CONSTRUÇÃO DE UM HEAP DE MÁXIMO

- Problema computacional

Construir Heap
Instância: (v, n) , onde v é um vetor indexado de 1 a n
Resposta: os elementos de v reorganizados como heap.

- Observe as folhas individualmente.
- Cada folha é raiz de uma subárvore sem descendentes.
- Logo, cada uma das **folhas** é um **heap de máximo**.

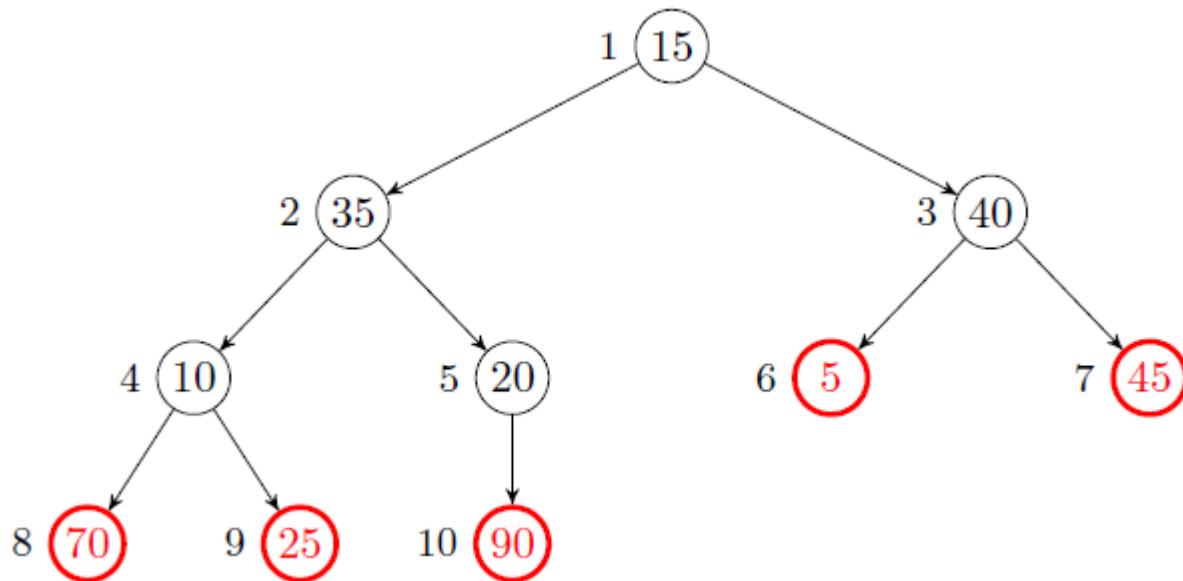
1	2	3	4	5	6	7	8	9	10
15	35	40	10	20	5	45	70	25	90



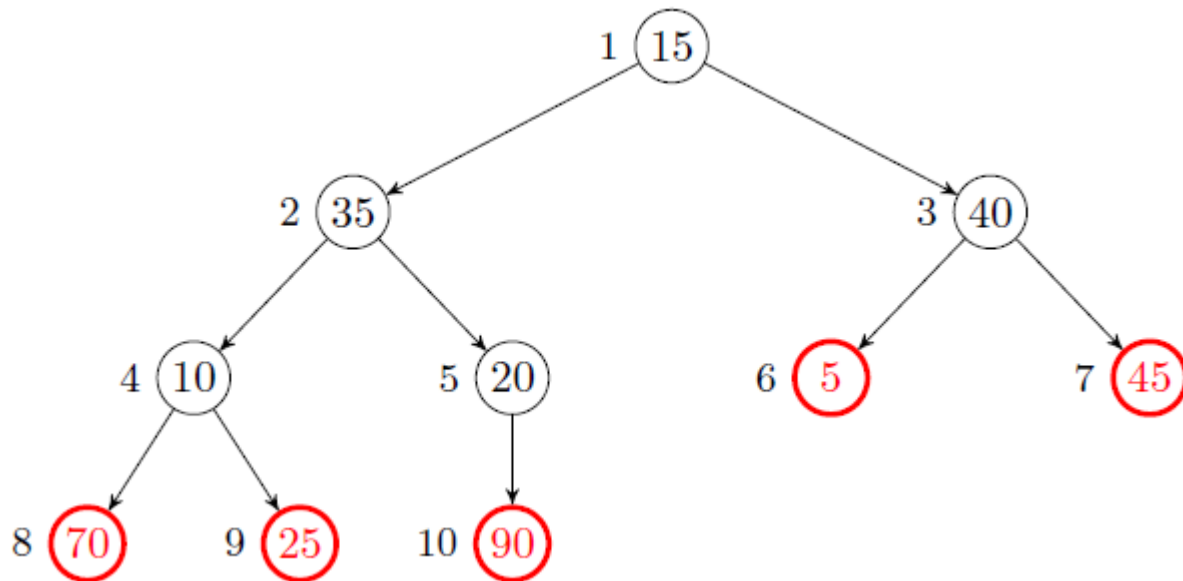
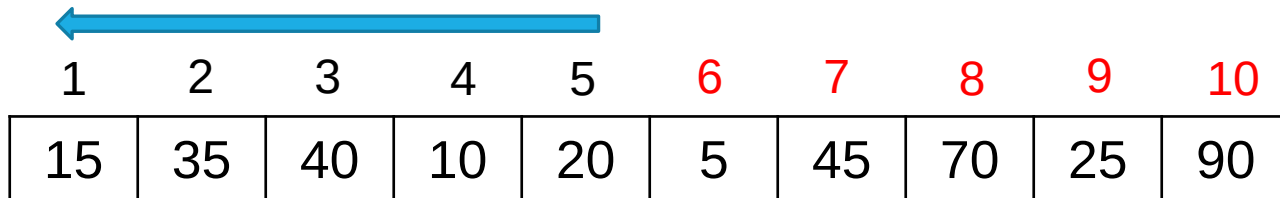
Onde as folhas se encontram no vetor?

$$\left\lfloor \frac{n}{2} \right\rfloor + 1$$

1	2	3	4	5	6	7	8	9	10
15	35	40	10	20	5	45	70	25	90

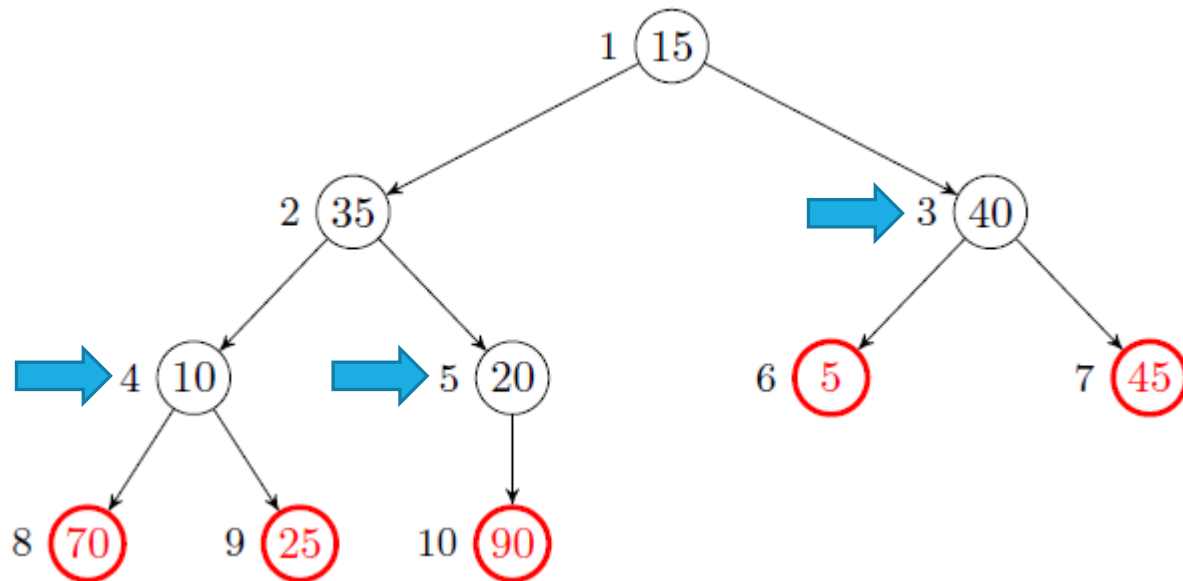


Qual é a ideia do algoritmo de construção do heap?



- Observe os nós rotulados com índices 3, 4 e 5.
- Cada um deles tem como filhos raízes de *heaps* de máximo.
- Esses nós satisfazem a condição de entrada do *max-heapify*.

1	2	3	4	5	6	7	8	9	10
15	35	40	10	20	5	45	70	25	90



CONSTRUÇÃO DE UM HEAP DE MÁXIMO

- Sobre o algoritmo...
 - Será denominado de *build-max-heap*.
 - Será executado de trás para frente (a partir da metade do vetor), aplicando o algoritmo *max-heapify*.
 - Ao final, teremos um ***heap* de máximo**.

CONSTRUÇÃO DE UM HEAP DE MÁXIMO

- Algoritmo iterativo

build-max-heap(v, n)
para i de $\lfloor n/2 \rfloor$ até 1 passo -1
max-heapify(v, i)
Devolva v

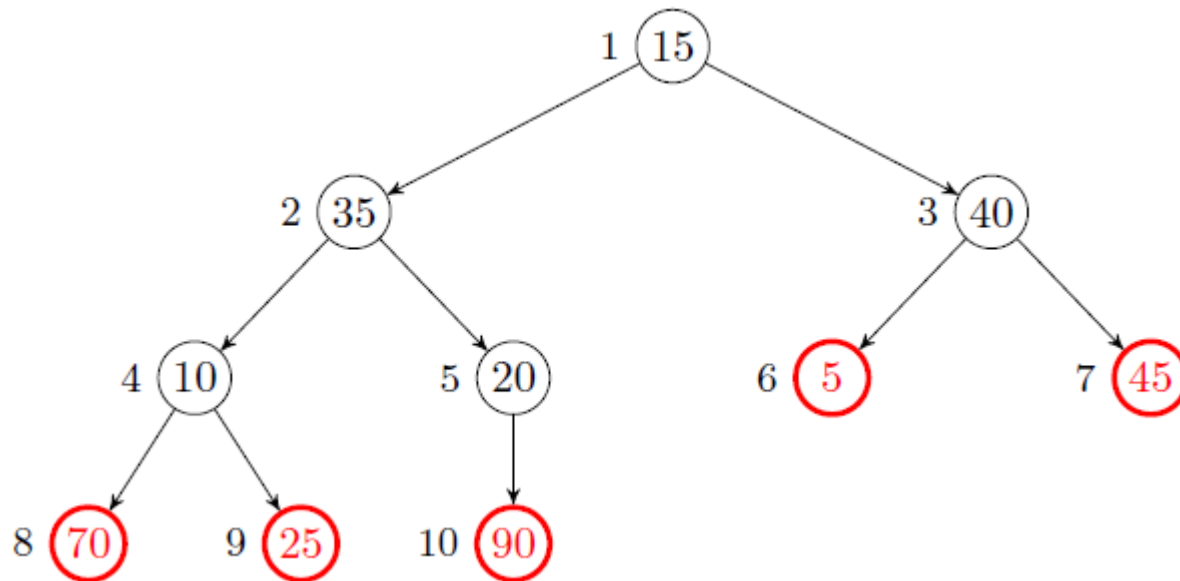
Antes de executar o
algoritmo build-max-heap

build-max-heap(v , n)

para i de $\lfloor n/2 \rfloor$ até 1 passo -1
 max-heapify(v , i)

Devolva v

1	2	3	4	5	6	7	8	9	10
15	35	40	10	20	5	45	70	25	90



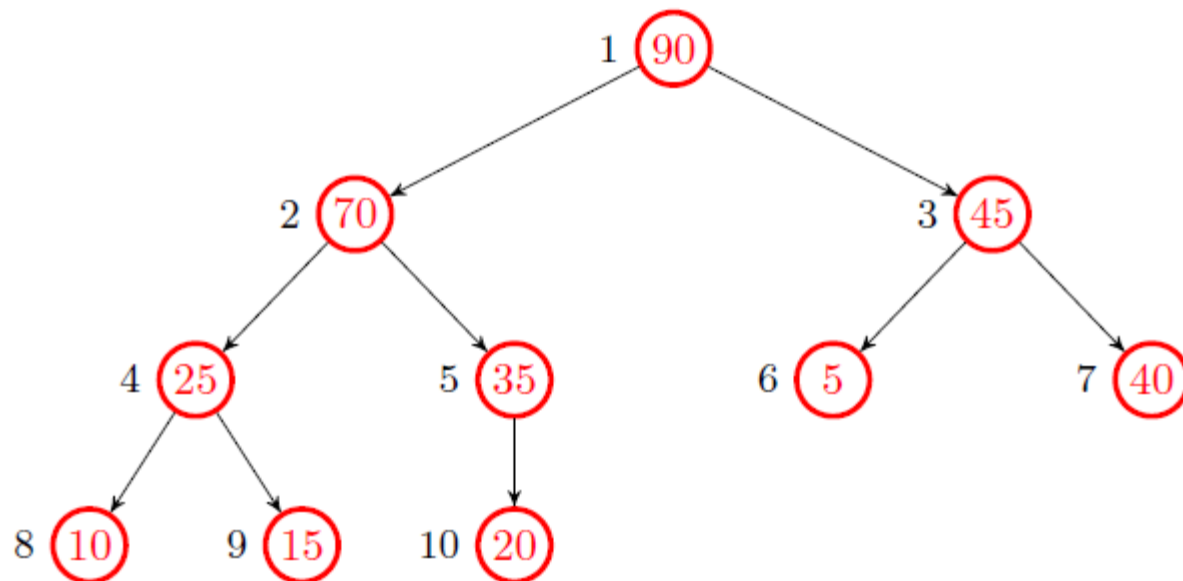
Depois de executar o algoritmo build-max-heap

build-max-heap(v, n)

para i de $\lfloor n/2 \rfloor$ até 1 passo -1
 max-heapify(v, i)

Devolva v

1	2	3	4	5	6	7	8	9	10
90	70	45	25	35	5	40	10	15	20



ANÁLISE

build-max-heap(v, n)

para i de $\lfloor n/2 \rfloor$ até 1 passo -1
 max-heapify(v, i)

Devolva v

- Análise do pior caso

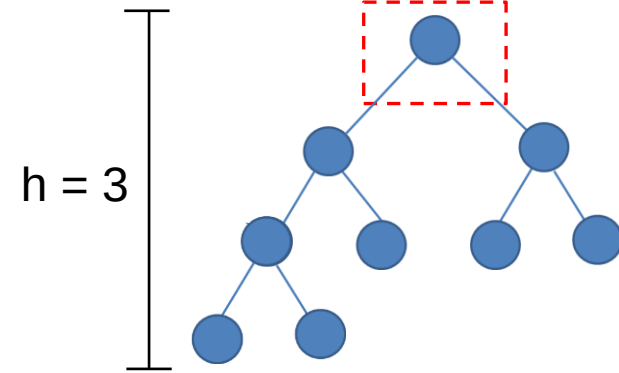
$C(n)$: número de comparações entre os elementos do vetor de tamanho n .

build-max-heap(v, n)
para i de $\lfloor n/2 \rfloor$ até 1 passo -1
max-heapify(v, i)
Devolva v

RACIOCINANDO...

- O laço será executado exatamente $\lfloor n/2 \rfloor$ vezes.
- Cada execução chama *max-heapify*, tem como pior caso $c_{mh}^+(n) = 2 \cdot \lfloor \log_2 n \rfloor$ (aula passada).
- Dessa forma, pode-se pensar que o pior caso do algoritmo *build-max-heap* é $c^+(n) = n \cdot \lfloor \log_2 n \rfloor$
- No entanto, esse não é o pior caso do algoritmo.

RACIOCINANDO...



- O custo do algoritmo *max-heapify* depende da altura da árvore.
 - O pior caso ocorre apenas quando a chamada envolve a raiz principal.
 - Mas a quantidade de nós, a partir de onde vamos chamar o algoritmo, decresce exponencialmente conforme mudamos de nível.
- Logo, **apenas uma chamada** do algoritmo **max-heapify** vai custar o pior caso.

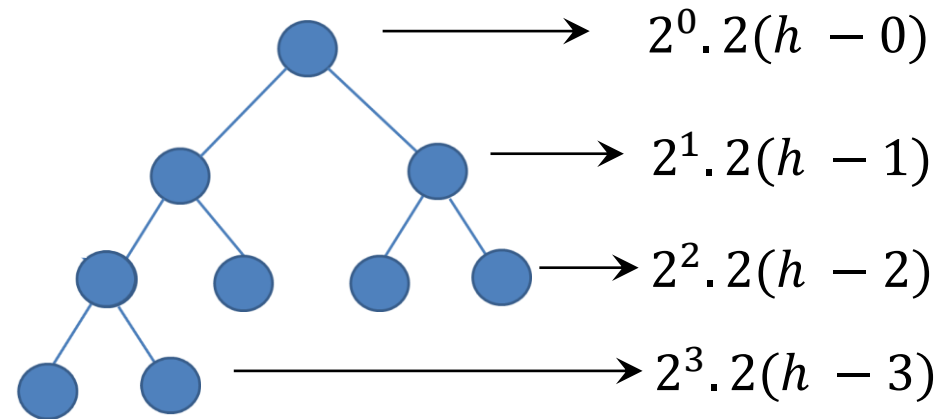
ANÁLISE DE PIOR CASO

Teorema. *O número máximo de comparações entre elementos do vetor que o algoritmo build-max-heap faz é $4n$.*

Demonstração. A quantidade de nós no **nível i** é 2^i e, para cada nó, a execução de max-heapify custa $2(h - i)$ no pior caso.

Demonstração. A quantidade de nós no nível i é 2^i e, para cada nó, a execução de max-heapify custa $2(h - i)$ no pior caso. Assim basta **somar todos os níveis**.

$$C^+(n) = 2^0 2(h - 0) + 2^1 2(h - 1) + 2^2 2(h - 2) + \dots + 2^h 2(h - h)$$



Demonstração. A quantidade de nós no nível i é 2^i , e para cada um gasta-se $h - i$ passos no pior caso para executar o algoritmo max-heapify. Assim basta somar todos os níveis.

$$\begin{aligned} C^+(n) &= 2^0 2(h - 0) + 2^1 2(h - 1) + 2^2 2(h - 2) + \dots + 2^h 2(h - h) \\ &= 2(2^0(h - 0) + 2^1(h - 1) + 2^2(h - 2) + \dots + 2^h(h - h)) \\ &= 2 \sum_{i=0}^h 2^i (h - i) \\ &= 2 \left(\sum_{i=0}^h 2^i h - \sum_{i=0}^h 2^i i \right) = 2 \left(h \sum_{i=0}^h 2^i - \sum_{i=0}^h 2^i i \right) \end{aligned}$$

$$\sum_{i=0}^h 2^i$$

OBSERVAÇÃO 1

Teorema 1. *Em uma árvore binária completa, $n = 2^{h+1} - 1$*

Lembrando, a fórmula da soma de p.g. finita é:

$$\sum_{i=0}^k c^i = \frac{c^{k+1} - 1}{c - 1}$$

$$\sum_{i=0}^k 2^i = \frac{2^{h+1} - 1}{2 - 1} = 2^{h+1} - 1$$

$$\sum_{i=0}^h 2^i i$$

OBSERVAÇÃO 2

Teorema 2.
$$\sum_{i=0}^k 2^i \cdot i = k2^{k+2} - (k+1)2^{k+1} + 2$$

Demonstração: Lembrando, a fórmula da soma de p.g. finita é:

$$\sum_{i=0}^k c^i = \frac{c^{k+1} - 1}{c - 1}$$

Derivando os dois lados e multiplicando por c (não veremos os detalhes)

$$\sum_{i=0}^k c^i i = \frac{kc^{k+2} - (k+1)c^{k+1} + c}{(c-1)^2}$$

Fazendo $c = 2$, chegamos no resultado desejado

- Vamos substituir as seguintes equações:

$$\sum_{i=0}^h 2^i = 2^{h+1} - 1 \quad \text{e} \quad \sum_{i=0}^k 2^i \cdot i = k2^{k+2} - (k+1)2^{k+1} + 2$$

$$C^+(n) = 2^0 2(h-0) + 2^1 2(h-1) + 2^2 2(h-2) + \dots + 2^h 2(h-h)$$

$$= 2 \left(h \sum_{i=0}^h 2^i - \sum_{i=0}^h 2^i i \right)$$

- Assim, substituindo...

$$C^+(n) = 2(h(2^{h+1} - 1) - (h2^{h+2} - (h+1)2^{h+1} + 2))$$

$$C^+(n) = 2^0 2(h - 0) + 2^1 2(h - 1) + 2^2 2(h - 2) + \dots + 2^h 2(h - h)$$

$$= 2 \left(h \sum_{i=0}^h 2^i - \sum_{i=0}^h 2^i i \right)$$

$$C^+(n) = 2(h(2^{h+1} - 1) - (h2^{h+2} - (h+1)2^{h+1} + 2))$$

$$= 2(h2^{h+1} - h - h2^{h+2} + h2^{h+1} + 2^{h+1} - 2)$$

$$= 2(2h2^{h+1} - h - h2^{h+2} + 2^{h+1} - 2)$$

$$= 2(h2^{h+2} - h - h2^{h+2} + 2^{h+1} - 2)$$

$$= 2(2^{h+1} - h - 2)$$

$$= 2(2^{h+1}) - 2(h + 2)$$

$$\leq 2(2^{h+1})$$

$$= 4 \cdot 2^h = 4 \cdot 2^{\lfloor \log_2 n \rfloor} \leq 4 \cdot 2^{\log_2 n} = 4n$$

Como $h = \lfloor \log_2 n \rfloor$

PRÓXIMA AULA

- Algoritmo de ordenação heapsort.